

Polymorphism and Inheritance in Object-Oriented Programming

Zadid Bin Azad

Department of Computer Science and Engineering
Islamic University of Technology
zadidbinazad@iut-dhaka.edu

June 6, 2026

Teaching Insight

A few boxes in this guide are coloured differently and titled **Question to Carry Forward**. These are *not* answered on the page. Each carries a label such as (Q1) and a single clue. Read each open question when you reach it, write down your own attempt. The reasoning you do in that gap is the point of the exercise.

1 Introduction: You Have Already Done This

Polymorphism is one of the four canonical pillars of object-oriented programming, alongside encapsulation, abstraction, and inheritance. The word is Greek for “many forms,” and in introductory courses it is often presented as something exotic that arrives late in the syllabus. In fact, most students have already used it without being told its name.

Consider a class you have certainly written:

```
1 public class Employee {  
2     private String name;  
3  
4     public Employee(String name) {  
5         this.name = name;  
6     }  
7  
8     @Override  
9     public String toString() {  
10         return "Employee(" + name + ")";  
11     }  
12 }
```

Listing 1: Overriding a method that every class already has.

Every class in Java silently inherits a `toString` method from the universal parent class `Object`. When you wrote your own `toString`, you did not invent a new method; you *replaced* the inherited one with a version of your own. This act of replacing an inherited method with a more specific version has a name in object-oriented design: *overriding*. And overriding is not a topic of its own — it is one of the two faces of polymorphism.

Question for the Class

When you call `System.out.println(someObject)`, Java calls `toString` on that object. But `println` was compiled long before your class existed — it has never heard of `Employee`. How does it end up calling *your* `toString` and not the default one from `Object`?

Hold on to that question; the whole second half of this guide is built to answer it. For now, accept the headline: the behaviour you call by one name (`toString`) takes “many forms” depending on the object it is called on. That is polymorphism, and you have been using it all along.

Working It Through

Polymorphism is the ability of a single name — a method name or an operator — to stand for more than one concrete behaviour. The right behaviour is selected for you, either by the compiler or by the running program. The rest of this guide is the story of *which* selector does the choosing, and *when*.

2 The Running Scenario: A Payroll System

Consider a company that needs software to pay its staff. The business rules include the following:

- every employee has a name and an identification number, fixed when they join;
- every employee can have their pay computed, but the *rule* for computing pay differs by role: a salaried manager, an hourly developer, and a commissioned salesperson are each paid by a different formula;
- the payroll department wants to process a mixed list of all staff uniformly — “for each employee, compute this month’s pay” — without writing a separate loop per role;
- there is no such thing as a “generic employee” on the books: every real employee has a specific role, and a plain employee with no pay rule should never be created;
- shared data, such as the base monthly figure each role works from, must be usable by the role-specific code but must remain hidden from code outside the payroll hierarchy.

These rules will reappear, one at a time, and each will force a design decision that reveals a facet of polymorphism or inheritance.

3 Two Flavours of Polymorphism: Compile-time and Runtime

Polymorphism in Java comes in two distinct forms, and the cleanest way to tell them apart is to ask a single question: *who decides which behaviour runs, and when do they decide it?*

- **Compile-time polymorphism.** The choice is made by the *compiler*, while it is still translating your source code, before the program ever runs. Because the decision is frozen at compile time, it can depend only on information the compiler can see: the number and types of the arguments you wrote at the call site. In Java this form is realised by *method overloading* (and operator behaviour such as `+` meaning both addition and string concatenation).
- **Runtime polymorphism.** The choice is deferred until the program is *running*, and is made by the Java Virtual Machine based on the *actual type of the object* at the moment of the call. The compiler cannot make this choice, because the object that a variable refers to may not

be decided until runtime. In Java this form is realised by *method overriding* — exactly what you did to `toString`.

The contrast is sharpest in code. First, compile-time selection by overloading:

```

1 public class Payroll {
2     public void log(double amount) {
3         System.out.println("Amount: " + amount);
4     }
5     public void log(double amount, String reason) {
6         System.out.println("Amount: " + amount + " (" + reason + ")");
7     }
8 }
9
10 // At the call site:
11 payroll.log(5000.0);           // compiler picks log(double)
12 payroll.log(5000.0, "bonus"); // compiler picks log(double, String)

```

Listing 2: Compile-time polymorphism: one name, several signatures.

When the compiler reads `payroll.log(5000.0, "bonus")`, it counts the arguments and inspects their types, finds the one method whose parameter list matches, and writes that exact choice into the compiled program. Nothing about this decision can change while the program runs.

Now runtime selection by overriding:

```

1 Object thing = makeSomething(); // returns an Employee, a String, ... decided
   at runtime
2 System.out.println(thing);      // which toString runs?

```

Listing 3: Runtime polymorphism: one call, behaviour chosen by the live object.

Here the compiler genuinely cannot know what `makeSomething()` will hand back on any given run. The decision about *which* `toString` executes is therefore postponed until the line actually runs and the real object is in hand. The JVM looks at that object and calls the version of `toString` belonging to its real class.

Question for the Class

In the overloading example, suppose you delete the second `log` method and call `payroll.log(5000.0, "bonus")`. At what moment do you learn something is wrong — while typing/compiling, or only when the program runs? Now ask the same about a misspelled override. What does your answer tell you about which form is checked “early” and which “late”?

Working It Through

Overloading is resolved *early*, by the compiler, from the static text of the call. Deleting an overload is a compile error — you find out immediately. Overriding is resolved *late*, by the JVM, from the runtime object. The two forms answer different questions: overloading asks “which method shape did the programmer mean here?”; overriding asks “which class does this object really belong to right now?”

4 Compile-time Polymorphism, in Detail

Overloading lets several methods share one name as long as their *parameter lists* differ — in number of parameters, in their types, or in their order. The compiler treats the name together with the parameter list as the method's true identity (its *signature*) and uses the signature at each call site to pick exactly one method.

What does this buy us? At first glance, nothing: we could have named the methods `logAmount`, `logAmountWithReason`, and so on. The gain is conceptual, not mechanical. Overloading lets you give *the same name to the same idea*, and vary only the details of the input. The reader of `payroll.log(...)` sees one operation — “record this” — and does not have to memorise a family of near-identical names. You have already met this in constructor overloading: several constructors, one class name, each a different legitimate way to build the object. Method overloading is the same move applied to ordinary methods.

```
1 public class Money {
2     public Money add(Money other)           { /* combine two Money values */ }
3     public Money add(double rawAmount)      { /* add a plain number */ }
4     public Money add(double a, double b)    { /* add two numbers at once */ }
5 }
```

Listing 4: Overloads describe one idea with different inputs.

Teaching Insight

A reliable way to make the “compile-time” nature concrete is to put two overloads on the projector, then change a literal argument from 5000 (an `int`) to 5000.0 (a `double`) and ask the class to predict which overload the compiler now selects. The prediction can be made purely by reading the code, with the program never running — which is exactly the definition of compile-time resolution.

Question to Carry Forward (Q1)

Two methods in the same class are written with the same name and the same parameter list, differing only in their return type — one returns `int`, the other `double`. Is that a valid pair of overloads?

Clue: the compiler chooses an overload by reading the call site. Look at a call like `x = obj.compute(a, b);` and ask yourself what parts of that line the compiler is allowed to use to make its choice — and whether the return type is one of them.

5 Runtime Polymorphism Needs a Prerequisite: Subtypes

Runtime polymorphism promised something the compiler cannot deliver: a single call site that runs different code depending on the live object. But step back and ask what that even requires. For the same line `x.calculatePay()` to run a manager's formula on one iteration and a developer's on the next, two things must hold at once:

1. the variable `x` must be allowed to refer, at different times, to objects of different classes; and
2. all those classes must agree to understand the call `calculatePay()`, even though each answers it differently.

That agreement is precisely what a *subtype* relationship provides. If `Manager` and `Developer`

are both *subtypes* of `Employee`, then a variable declared as `Employee` may legally hold either of them, and every one of them is guaranteed to understand any call that `Employee` understands. Without such a relationship there is nothing to substitute and nothing to switch between: the compiler would reject `Employee x = new Manager();` outright, and the question of “which version runs” would never arise.

Working It Through

Runtime polymorphism is substitution in action. It needs a family of types that are interchangeable through a common parent type — subtypes — so that one variable can stand for any member of the family while a single call name means “do the right thing for whatever you actually are.” Our task, therefore, is to learn how to create subtypes. There is more than one mechanism; we begin with the first and most direct one: *inheritance*.

6 Inheritance: Parent and Child Classes

Inheritance lets one class be defined as a specialised version of another. The existing class is the *parent* (also called the superclass or base class); the new, more specific class is the *child* (subclass or derived class). In Java the relationship is declared with the keyword `extends`.

```
1 public class Employee {           // parent
2     protected String name;
3     protected int id;
4
5     public Employee(String name, int id) {
6         this.name = name;
7         this.id = id;
8     }
9
10    public String describe() {
11        return id + ": " + name;
12    }
13 }
14
15 public class Manager extends Employee { // child
16     private int teamSize;
17     // Manager automatically has name, id, and describe()
18 }
```

Listing 5: A child class declares its parent with `extends`.

The child receives, without rewriting them, the accessible members of the parent. A `Manager` already has a `name`, an `id`, and a working `describe()` method, even though none of those appear in the `Manager` source. This is the literal sense in which the child “inherits” from the parent.

What exactly is inherited deserves care:

- **Public and protected fields and methods** are inherited and are directly usable inside the child.
- **Private members** technically still exist inside every child object — a `Manager` object carries a `name` field even if it were private — but the child cannot *name* them directly; they remain the parent’s private business.
- **Constructors are not inherited.** A child does not receive its parent’s constructors as its own. (How a child is built is the subject of the next section.)

- **Static members** belong to the class, not to instances, and are not part of this object-level inheritance story.

Question for the Class

A `Manager` is a kind of `Employee`, so a `Manager` object clearly needs a `name` and an `id`. But the `Manager` class body never declares those fields. Where do they come from, and which constructor is responsible for giving them their initial values?

7 The `super` Keyword

Because constructors are not inherited, a child cannot simply assume its parent's fields are ready to use. Before a `Manager` can be considered a valid `Employee`, the `Employee` part of it must be initialised. Java enforces this with a rule: **every child constructor must, as its very first action, run one of its parent's constructors.** The tool for doing so is `super(...)`.

```
1 public class Manager extends Employee {
2     private int teamSize;
3
4     public Manager(String name, int id, int teamSize) {
5         super(name, id);           // build the Employee part first
6         this.teamSize = teamSize;
7     }
8 }
```

Listing 6: A child constructor initialises the parent part first, via `super`.

The call `super(name, id)` hands those two values to the `Employee` constructor, which sets the inherited `name` and `id` fields. Only after the parent part is fully built does the child finish its own work. If you omit the `super(...)` call, Java silently inserts a call to the parent's *no-argument* constructor — and if the parent has no such constructor, the code will not compile, which is a common first surprise.

`super` has a second use: from inside a child method, `super.someMethod()` calls the parent's version of a method that the child has overridden. This lets a child *extend* rather than fully replace inherited behaviour:

```
1 public class Manager extends Employee {
2     @Override
3     public String describe() {
4         return super.describe() + " (manages " + teamSize + ")";
5     }
6 }
```

Listing 7: `super.method()` reuses the parent's version, then adds to it.

8 protected Access: Encapsulation Inside a Hierarchy

You already know `private` hides a member from everyone outside the class — and that “everyone” includes child classes. A strictly private field in `Employee` would be invisible to `Manager`, which is sometimes too strict: a child often legitimately needs to work with the parent's data.

Java offers a middle level for exactly this situation: **protected**. A protected member is visible to the class itself and to its children (and to other classes in the same package), but remains

hidden from unrelated outside code. It widens the circle of trust to descendants without opening the field to the whole program.

```

1 public class Employee {
2     protected double baseMonthly;    // children may read/use this directly
3
4     public Employee(double baseMonthly) {
5         this.baseMonthly = baseMonthly;
6     }
7 }
8
9 public class Developer extends Employee {
10     public double calculatePay() {
11         return baseMonthly * 1.0;    // allowed: a child can see protected
12                                     // fields
13     }
14 }
15
16 // Elsewhere, in unrelated code:
17 // someEmployee.baseMonthly = -1;    // rejected: protected is not public

```

Listing 8: protected shares with children but not with the outside world.

Seen this way, `protected` is encapsulation that respects the family tree. The outside world still cannot reach in and corrupt `baseMonthly`; the invariant that pay figures stay sensible is still defended at the boundary of the hierarchy. What changes is that the responsibility for honouring that invariant is now shared between the parent and every child, because every child can touch the field directly.

Question to Carry Forward (Q2)

With `private`, only one class — the one that wrote the field — can ever break its invariant, so there is exactly one file to audit. With `protected`, who else can now write to the field directly? If a bug lets `baseMonthly` go negative, how many files might you have to read to find the culprit, and how does that compare with `private`?

Clue: count the classes that are allowed to assign to the field in each case. Encapsulation is partly about shrinking the set of code that could possibly be responsible for a broken value.

9 Why Inheritance at All? Reuse, and the Limits of Composition

The original motivation for inheritance was *code reuse*. Shared data and shared behaviour can be written once, in a parent, and every child gets them for free. `Employee` defines `name`, `id`, and `describe()` a single time; `Manager`, `Developer`, and `Salesperson` all benefit without repetition.

But a thoughtful student should object immediately: *we already have a tool for reuse*. We can give one class a field whose type is another class, and call that object's methods. This is *composition* — building a class out of other objects it holds. We have used it freely: an account “has a” currency, an order “has a” list of items.

```

1 public class Employee {
2     private Address homeAddress;    // an Employee HAS-A Address
3
4     public String city() {
5         return homeAddress.getCity();    // reuse Address's behaviour
6     }
7 }

```

Listing 9: Composition: reuse by holding another object as a field.

So if composition already reuses code, why was inheritance ever needed? The answer is the one thing composition does *not* give you: it does not make the holder a *subtype* of the thing it holds. An `Employee` that holds an `Address` is in no sense “a kind of `Address`,” and you could never store an `Employee` in a variable of type `Address`. Composition reuses behaviour; inheritance reuses behaviour *and* establishes the parent–child subtype relationship — the very prerequisite we found that runtime polymorphism requires. That is the capability inheritance adds on top of plain reuse.

This gives a practical thinking tool for deciding between the two. Say the relationship aloud:

- If “a B **is** a A” is true — a `Manager` is an `Employee`, a `Developer` is an `Employee` — then B can extend A. The child genuinely is a specialised kind of the parent and can stand in for it.
- If “a B **has** a A” is true — an `Employee` has an `Address`, a `Manager` has a team — then B should hold A as a field. The relationship is ownership, not kinship.

Where Students Typically Slip

Students reach for inheritance whenever it would save typing, even when the relationship is really “has-a.” A classic slip is making `Manager` `extends` `Team` (or `Stack` `extends` `ArrayList`) because the parent happens to contain useful methods. The test is not “would I get free methods?” but “is this object honestly a *kind of* that?” If you cannot say “is-a” with a straight face, the free methods are a trap: the child now exposes inherited operations that make no sense for it.

Question to Carry Forward (Q3)

A `Manager` *is* an `Employee`, and a `Manager` also *leads a team of* `Employees`. One of these relationships should be modelled with `extends` and the other with a field. Decide which is which. Then imagine someone modelled “leads a team of” with inheritance instead — what absurd statement about a `Manager` would the type system then believe?

Clue: translate each relationship into an “is-a” / “has-a” sentence first. Then recall that `extends` claims the child can be used *anywhere* the parent can.

10 Multi-level Hierarchies

A child can itself be a parent. Inheritance chains to any depth, and each level adds or refines what it received from above.

```

1 public class Employee {
2     protected String name;
3     public Employee(String name) { this.name = name; }
4 }
5
6 public class Manager extends Employee {
7     protected int teamSize;
8     public Manager(String name, int teamSize) {
9         super(name); // build the Employee part
10        this.teamSize = teamSize;
11    }
12 }
```

```
13
14 public class Director extends Manager {
15     private int divisions;
16     public Director(String name, int teamSize, int divisions) {
17         super(name, teamSize);           // build the Manager part (which builds
18         // Employee)
19         this.divisions = divisions;
20     }
21 }
```

Listing 10: A three-level chain; each constructor builds the level above it.

A `Director` *is a* `Manager`, which *is an* `Employee`. Constructing a `Director` triggers a chain of `super(...)` calls all the way up to `Employee`, so the object is assembled from the top of the family tree downward. When you call a method on a `Director`, Java starts at the `Director` class and walks *up* the chain until it finds a definition — so a `Director` can use a method defined three levels above it as if it were its own.

Question to Carry Forward (Q4)

You call `director.name` (or a method that only `Employee` defines) on a `Director` object, where `Director` extends `Manager` extends `Employee`. Java has to locate that member. Starting from where, and moving in which direction along the chain, does the search proceed — and what is the first match it accepts?

Clue: the object knows its most specific (actual) class. Think about which end of the chain that is, and why searching from the other end could pick the wrong version when a method has been overridden midway.

11 Abstract Classes

Return to a business rule from our scenario: there is no such thing as a “generic employee.” Every real person on the payroll is a manager, a developer, or a salesperson, each with its own pay formula. Yet our `Employee` class, as written, can be instantiated directly — `new Employee("Sam", 1)` compiles — producing exactly the meaningless object the business says should never exist. Worse, `Employee` has no honest way to implement `calculatePay()`: there is no formula that is correct for an employee whose role we do not know.

This is what *abstract classes* are for. We already know how to inherit from a concrete (ordinary) class, so why introduce a new kind? Because some parents exist only to be specialised: they collect the shared data and behaviour of a family, and they declare *what* every member must be able to do, while refusing to commit to *how*, and refusing to be created on their own.

Two keywords carry this:

- Marking the *class* **abstract** forbids direct instantiation: `new Employee(...)` becomes a compile error. The class can only be used by being extended.
- Marking a *method* **abstract** declares its signature with no body. The class is saying “every concrete descendant must supply this; I will not.” A class that contains even one abstract method must itself be declared abstract.

Question for the Class

If you can never write `new Employee(...)`, the `Employee` constructor can never be called by client code. So should an abstract class even have a constructor? If you say no, ask yourself how the inherited `name` and `id` fields of a freshly created `Manager` ever get their values.

Working It Through

An abstract class *should* have a constructor, even though you cannot call it with `new`. The reason is the chaining rule from earlier: when a concrete `Manager` is created, its constructor's first act is `super(...)`, which runs the abstract `Employee` constructor to initialise the inherited fields. The abstract constructor is never the *start* of object creation, but it is an essential *step* in it. Its job is to set up the shared state that every subclass relies on — so it takes exactly the parameters that shared state needs, and validates them, just like any other constructor.

12 Example: An Abstract Parent and Concrete Children

The listing below assembles the ideas. `Employee` is abstract: it holds the shared state, validates it in a constructor, provides a concrete `describe()`, and declares `calculatePay()` as abstract — a promise that every real employee fills in. `Manager` and `Developer` are concrete children that supply that formula; `Developer` also overrides `describe()` and reuses the parent's version through `super`.

```

1 public abstract class Employee {
2
3     protected final String name;
4     protected final int id;
5     protected double baseMonthly;
6
7     public Employee(String name, int id, double baseMonthly) {
8         if (name == null || name.isBlank())
9             throw new IllegalArgumentException("Name is required.");
10        if (baseMonthly < 0.0)
11            throw new IllegalArgumentException("Base pay cannot be negative.");
12        this.name = name;
13        this.id = id;
14        this.baseMonthly = baseMonthly;
15    }
16
17    // Declared, not defined: every concrete employee MUST provide a formula.
18    public abstract double calculatePay();
19
20    // Shared, concrete behaviour available to all employees.
21    public String describe() {
22        return id + ": " + name;
23    }
24
25    @Override
26    public String toString() {
27        return describe() + " -> pay " + calculatePay();
28    }
29 }
30
31 public class Manager extends Employee {
32     private final double bonus;

```

```

33
34     public Manager(String name, int id, double baseMonthly, double bonus) {
35         super(name, id, baseMonthly);    // initialise the shared Employee state
36         this.bonus = bonus;
37     }
38
39     @Override
40     public double calculatePay() {        // the Manager's formula
41         return baseMonthly + bonus;
42     }
43 }
44
45 public class Developer extends Employee {
46     private final int overtimeHours;
47     private final double hourlyRate;
48
49     public Developer(String name, int id, double baseMonthly,
50                     int overtimeHours, double hourlyRate) {
51         super(name, id, baseMonthly);
52         this.overtimeHours = overtimeHours;
53         this.hourlyRate = hourlyRate;
54     }
55
56     @Override
57     public double calculatePay() {        // a different formula
58         return baseMonthly + overtimeHours * hourlyRate;
59     }
60
61     @Override
62     public String describe() {
63         return super.describe() + " (developer)";    // extend, don't replace
64     }
65 }

```

Listing 11: An abstract base class with concrete subclasses.

Notice what each mechanism contributes. Inheritance lets `Manager` and `Developer` reuse `name`, `id`, `describe()`, and `toString()` without rewriting them. `super(...)` initialises the inherited state. The abstract `calculatePay()` guarantees that the parent's `toString()` can safely call a method it does not itself implement, trusting the child to have provided it. And crucially, `Manager` and `Developer` are now *subtypes* of `Employee`.

13 Closing the Loop: Subtyping Delivers Runtime Polymorphism

We set out needing subtypes so that one variable could stand for many kinds of object and one call name could mean “do the right thing.” Inheritance has now produced exactly that: `Manager` and `Developer` are subtypes of `Employee`, so a variable of type `Employee` can hold either, and both are guaranteed to understand `calculatePay()` and `toString()`.

```

1 Employee[] staff = {
2     new Manager("Ada", 1, 9000, 2000),
3     new Developer("Lin", 2, 7000, 10, 50),
4     new Manager("Sam", 3, 9500, 1500)
5 };
6
7 for (Employee e : staff) {        // declared type is Employee for all
8     System.out.println(e);        // calls toString -> calls calculatePay
9 }

```

Listing 12: One loop, one call name, behaviour chosen per object at runtime.

The variable `e` is declared as `Employee`, so the compiler only checks that `Employee` promises a `calculatePay()` and a `toString()` — which, as an abstract method and an inherited method, it does. But the compiler does *not* decide which version runs. On each iteration the JVM inspects the actual object `e` refers to — a `Manager` here, a `Developer` there — and dispatches to *that class's* overridden method. The manager's pay formula and the developer's pay formula both flow out of the identical line `System.out.println(e)`.

Working It Through

This is runtime polymorphism, and it is now fully explained. The subtype relationship created by inheritance lets one `Employee` reference hold any member of the family; method overriding gives each member its own version of a shared call; and late dispatch by the JVM selects the right version from the live object. This is also the exact mechanism behind the puzzle from the introduction: `println` never heard of your class, yet it calls your `toString`, because every object carries its own overridden version and the JVM finds it at the moment of the call.

Question to Carry Forward (Q5)

In the line `Employee e = new Manager(...)`; the declared type of `e` is `Employee`, whose `calculatePay()` has no body at all (it is abstract). Yet `e.calculatePay()` compiles and runs the manager's formula. Separate two distinct questions: (a) why does the *compiler* accept this call, looking only at the declared type? and (b) why does the *running program* execute the `Manager` version rather than nothing?

Clue: the compiler reasons about the *declared* type and asks only “is this call promised?”; the JVM reasons about the *actual* object and asks “which class are you really?” Two different selectors, acting at two different times — the same split that names the two flavours of polymorphism.

14 Synthesis and Classroom Discussion

By the end of the development, the class should have internalised the following.

1. Polymorphism means one name, many behaviours; you were already using it every time you overrode `toString`.
2. Compile-time polymorphism (overloading) is resolved early, by the compiler, from the argument shapes written at the call site.
3. Runtime polymorphism (overriding) is resolved late, by the JVM, from the actual type of the live object.
4. Runtime polymorphism is impossible without subtypes: a family of interchangeable types reachable through a common parent type.
5. Inheritance creates subtypes *and* reuses code. Composition reuses code too, but does not create a subtype — which is the capability that distinguishes “is-a” (extend) from “has-a” (hold a field).

6. **super** initialises the inherited part of an object and lets a child extend rather than discard inherited behaviour; **protected** shares state with descendants while keeping it from the outside world.
7. An abstract class collects shared state and declares required behaviour without allowing its own instantiation; it still needs a constructor, because subclass construction runs it through **super**.

A productive closing exercise is to hand students a different family — shapes with an area, notifications with a delivery step, bank accounts with an interest rule — and ask them to identify the shared parent, decide which behaviour must be abstract, and predict, for a mixed list of the family, which version of that behaviour each element will run.

We created subtypes here by having one class *extend* another, inheriting its code along the way. But notice that runtime polymorphism only ever needed the *promise* that an object understands a call — the `calculatePay()` guarantee — not the inherited implementation that came bundled with it. That raises a question we have not touched: can we declare such a promise, and create subtypes, *without* a parent class handing down any code at all? Java has a keyword for precisely that. It is

interface

and it is where we go next.
